"Marginalia" a hand-built educational search engine

Building a Specialised Search Engine with an Inverted Index

Domain: Educational Content

Group Member	Student ID
Younus (Group Leader)	0432220005101082
Autoshi Aunkita Phoebe	0432220005101081
Shahida Akter Rimu	0432220005101091
Nur A Zannat Ruba	0432220005101105

Course: CSE 426
Section: 8A
Instructor: Mrinmoy Das Akash (Lecturer & Course Coordinator, CSE)
Date of Submission: 8 June, 2026

1. Problem Definition & Dataset

1.1 Objective and Domain

The goal of this project is to design and implement a specialized search engine entirely from scratch, without relying on any pre-built search or indexing library. Rather than searching the whole web, the engine searches within a focused collection of documents drawn from a single domain. We chose educational content as our domain, because such material is text-rich, freely available, and varied enough to make ranking meaningful, while remaining coherent enough to return sensible results.

Our corpus is built from English Wikipedia articles across five educational categories: Mathematics, Physics, Computer Science, Machine Learning, and Biology. These subjects give the index a clear academic focus and a mix of overlapping and distinct vocabulary, which lets us demonstrate both ranked retrieval and Boolean (AND/OR) query behaviour.

1.2 Dataset

Documents were collected programmatically using the official Wikipedia (MediaWiki) API rather than by scraping raw HTML. The API returns clean plain text, which keeps preprocessing simple, and using it respects Wikipedia's usage policy and avoids the fragility of parsing page markup. Each document is stored as a record containing a unique document ID, the article title, its source URL, and the plain-text body.

After collection and filtering of very short stub pages, the final dataset contained 156 documents. When indexed, these produced 11,689 unique terms. The dataset is saved as docs.json, and the built index is persisted to disk so the search application does not need to rebuild it on every run.

Domain	Educational content
Source Wikipedia (MediaWiki) API	
Categories	Mathematics, Physics, Computer Science, Machine Learning, Biology
Documents indexed 156	

Unique terms in index	11,689
Fields per document id, title, url, text	

1.3 Challenges of Building the Search Engine (P4)

Building a search engine from first principles is a non-routine problem: there is no single correct design, and many practical difficulties only become visible once real data flows through the system. The main challenges we encountered were:

- Data quality and noise. Because we collected articles by category membership, the crawl also returned non-article pages such as user pages (e.g. "User:Rishi200809" and its sandbox), administrative or award pages (e.g. "Wohlfarth Lectureship"), and community portals (e.g. "CIML community portal"). These pages are not genuine educational content and add noise to the index. This highlighted that raw collected data almost always needs cleaning before it is trustworthy.
- Varying document length. Articles ranged from short entries to long, detailed pages. Long documents naturally contain more term occurrences, so without correction they would unfairly dominate the ranking simply by being long.
- Vocabulary mismatch. Users and documents do not always use the same word form ("running" vs "ran", "networks" vs "network"). The query and the index must be normalised in exactly the same way, or valid matches are missed.
- No ground-truth relevance labels. Unlike a supervised task, there is no labelled "correct answer" for a query, so relevance had to be judged qualitatively by inspecting whether the top-ranked results were sensible for each query.
- Implementing retrieval without libraries. The core data structure and the ranking had to be built and reasoned about manually, which required understanding why an inverted index is efficient rather than simply calling an existing tool.

2. Preprocessing & Inverted Index Design

2.1 Preprocessing Pipeline

Every document, and every query, passes through the same preprocessing pipeline. Using one shared pipeline is a deliberate design choice: if the query were normalised differently from the index, a query term could never match its stored form. NLTK is used only for this preprocessing
stage, in line with the project constraints. The pipeline performs the following steps, each chosen for a specific reason (P1):

🎬 Lowercasing — so that “Search” and “search” are treated as the same term. 🎬 Tokenisation — the text is split into word tokens using a regular expression that keeps runs of letters and digits and discards punctuation and symbols.

🎬 Stopword removal — extremely common words such as “the”, “is” and “of” carry little discriminating meaning and are removed to keep the index focused and smaller. 🎬 Single-character removal — isolated single characters rarely help retrieval and are dropped.

🎬 Stemming — the Porter stemmer reduces words to a common root (“running”, “runs” and “ran” all become “run”), so different forms of the same word match each other. This directly addresses the vocabulary-mismatch challenge.

2.2 Inverted Index Structure

A naive approach would scan every document for every query, which is slow. An inverted index reverses the relationship: instead of mapping each document to the words it contains, it maps each word to the documents that contain it. Searching then becomes a fast dictionary lookup rather than a full scan.

Our index maps each term to a dictionary of document IDs, and each document ID to the list of positions at which the term occurs in that document:

term → { document_id → [position1, position2, ...] }

Storing positions rather than only a count was a deliberate decision. It gives us the term frequency for free (the length of the position list), provides the location needed to build a readable snippet around the first match, and leaves the door open to future phrase queries. Alongside the main index we also store, for each document, its title, URL and total token count; the token count is later used to normalise ranking by document length. The completed index is serialised to disk so it can be reloaded instantly without reprocessing the corpus.

3. Query Processing & Ranking

3.1 Boolean Retrieval (AND / OR)

The engine accepts both single-word and multi-word queries. Each query is first passed through the same preprocessing pipeline as the documents, producing a list of normalised query terms. For each term, the engine looks up the set of documents that contain it, then combines those sets according to the chosen mode:

🎬 OR mode returns the union of the document sets — a document qualifies if it contains at least one query term. This is broader and returns more results.

- AND mode returns the intersection — a document qualifies only if it contains every query term. This is narrower and more precise.

3.2 Ranking with TF-IDF

Qualifying documents are ranked using the classic TF-IDF weighting scheme. For a query term t in a document d, with N documents in the collection:

tf = 1 + log10(frequency of t in d)

idf = log10(N / number of documents containing t) weight(t, d)

= tf × idf

A document's score is the sum of weight(t, d) over all matching query terms. Term frequency rewards documents that use a term often; inverse document frequency reduces the influence of terms that appear in many documents, so a rare, informative word counts for more than a common one. Finally each score is divided by the square root of the document's length, which prevents long documents from ranking highly simply because they contain more words — directly addressing the document-length challenge.

3.3 Snippets and Keyword Highlighting

For each returned document the engine shows the title, the source link, a relevance score, and a short snippet. The snippet is centred on the first position where a query term appears, and every matching word within it is wrapped for highlighting so the user can immediately see why the document matched.

3.4 Trade-offs: Accuracy vs Efficiency

TF-IDF with an inverted index is fast and predictable, and for keyword queries it produces intuitive rankings. Its main limitation is that it treats a query as a bag of independent words: it ignores word order and meaning, so "machine learning" and "learning machine" receive identical scores, and it cannot recognise that two different words mean the same thing. More accurate approaches (for example phrase matching using the stored positions, or semantic embeddings) would improve relevance but at the cost of greater complexity and slower queries. For a focused, keyword-based educational search engine, TF-IDF is a reasonable balance of accuracy and efficiency.

4. System Analysis & Discussion (P3)

4.1 Key Design Decisions

- Separating data collection, indexing and search into distinct stages. This keeps

each part simple to reason about and test, and means the slow collection step only runs once. 🎬 Using one shared preprocessing pipeline for documents and queries, guaranteeing that query terms are comparable with indexed terms.

🎬 Storing positions in the postings list, which provides term frequency, snippet locations, and a path to phrase queries from a single structure.

🎬 Normalising TF-IDF scores by document length, so ranking reflects relevance rather than document size.

🎬 Persisting the built index to disk, so the web application starts and answers queries almost instantly.

4.2 Limitations

🎬 The corpus contains some non-educational noise pages that passed through category collection and were not filtered out.

🎬 Bag-of-words TF-IDF ignores word order and semantics, so phrasing and synonyms are not handled.

🎬 Stemming is sometimes aggressive and can merge words that are not truly related.

🎬 The index is held in memory, which suits a few hundred to a few thousand documents but would not scale to web-sized collections.

4.3 Possible Improvements

🎬 Filter the crawl to genuine article pages only, removing user, portal and administrative pages, to raise corpus quality.

🎬 Add phrase-query support using the positions already stored in the index. 🎬 Introduce field weighting so a match in the title counts more than a match in the body. 🎬 Experiment with lemmatisation instead of stemming for cleaner normalisation. 🎬 Add query suggestions or spelling correction to handle user typing errors.

5. Results & Conclusion

5.1 Index Statistics

Building the index over the full corpus produced the following statistics, confirming that the inverted index was constructed successfully:

Documents indexed	156
Unique terms 11,689	

Typical query time	~50 milliseconds

5.2 Sample Query Results

The table below summarises live queries run against the system. The contrast between OR and AND for a multi-word query clearly demonstrates that Boolean retrieval is working: the multi word query matches far fewer documents under AND than under OR, while a single-word query is identical in both modes (as expected, since AND and OR are the same for one term).

Query	Mode	Matching	Time
neural network	OR	**documents** 55	51 ms
neural network	AND	28	55 ms
neural	OR	31	51 ms
neural	AND	31	57 ms

5.3 System Screenshots

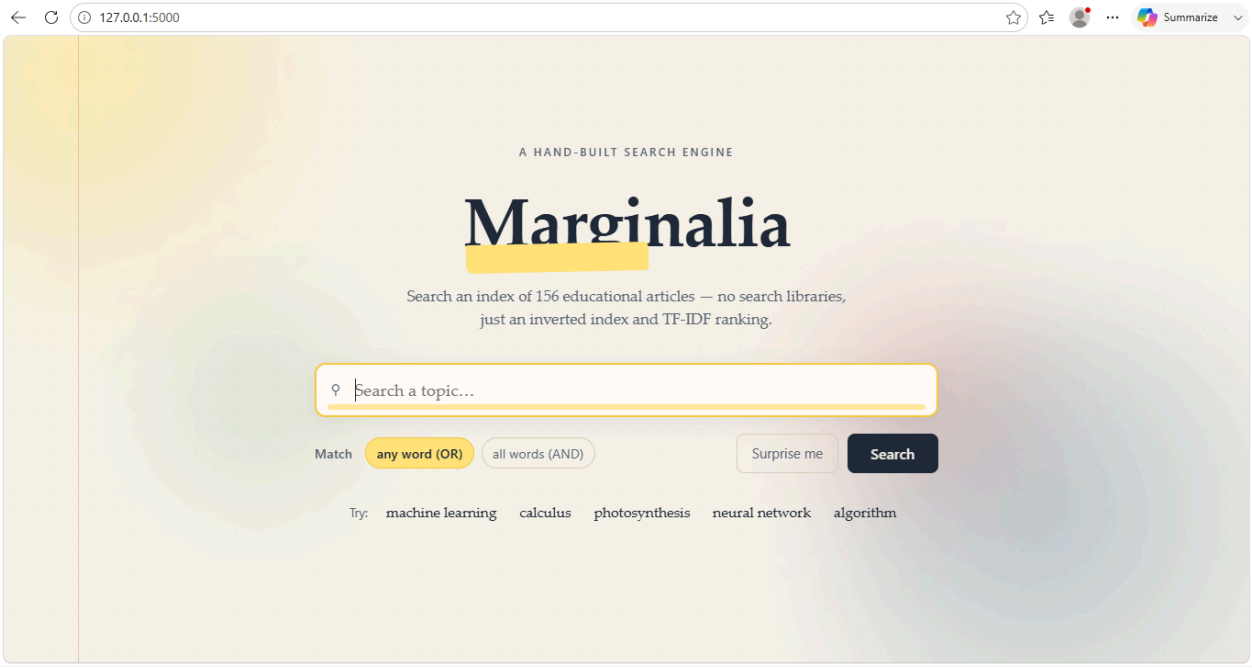

Figure 1: The Marginalia homepage, searching an index of 156 educational articles.

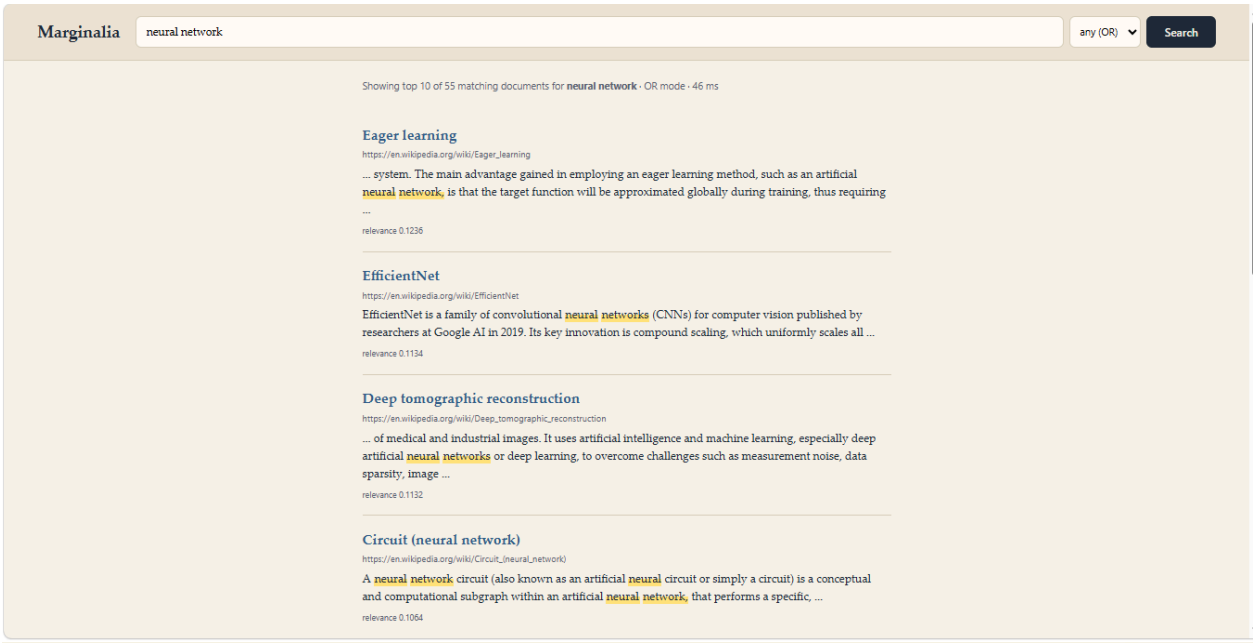

Figure 2: Multi-word query "neural network" in OR mode — 55 matching documents.

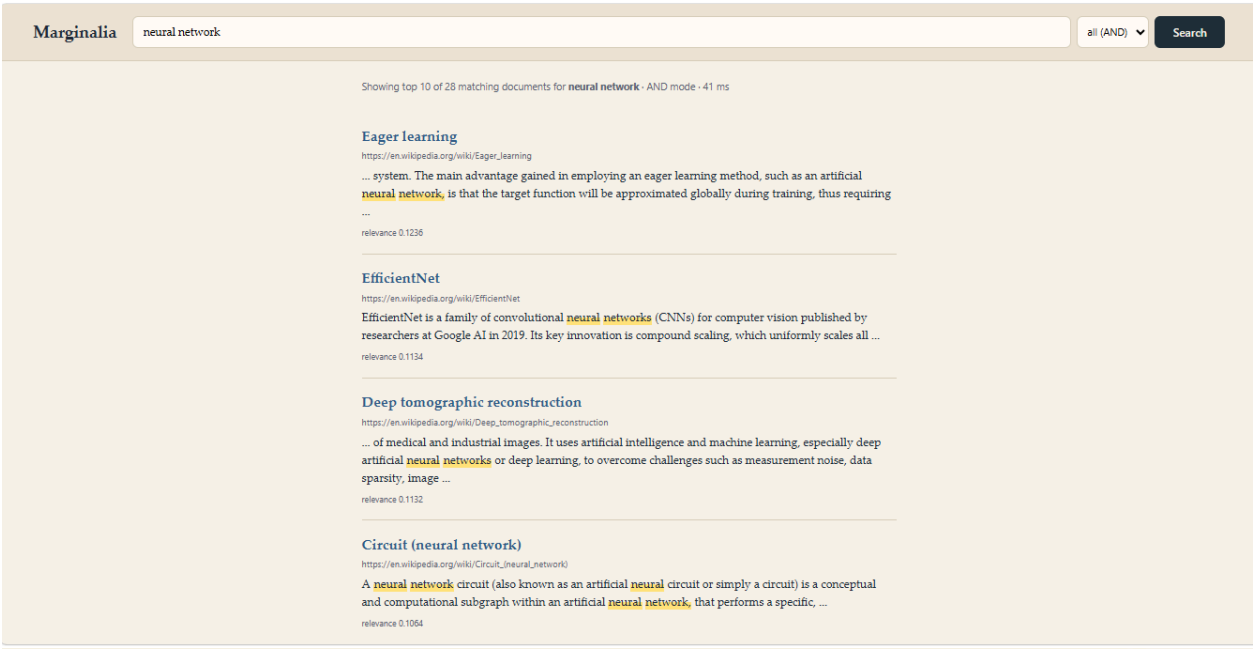

Figure 3: The same query in AND mode — 28 matching documents, showing Boolean filtering.

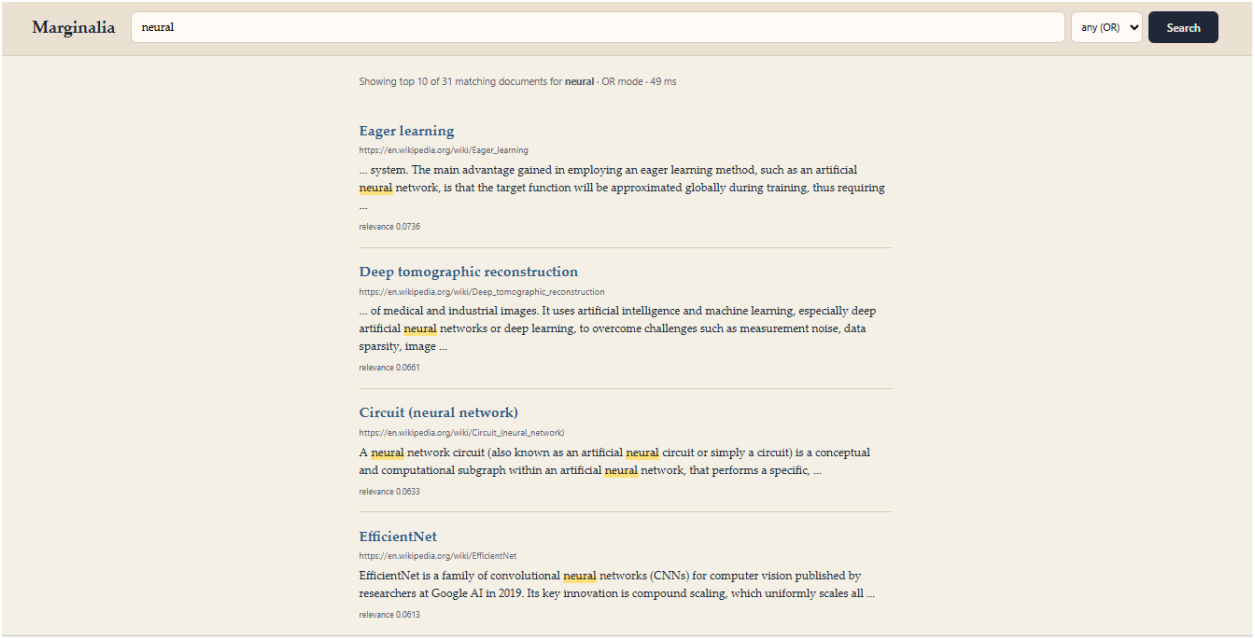

Figure 4: Single-word query "neural" in OR mode — 31 matching documents, with highlighted keywords.

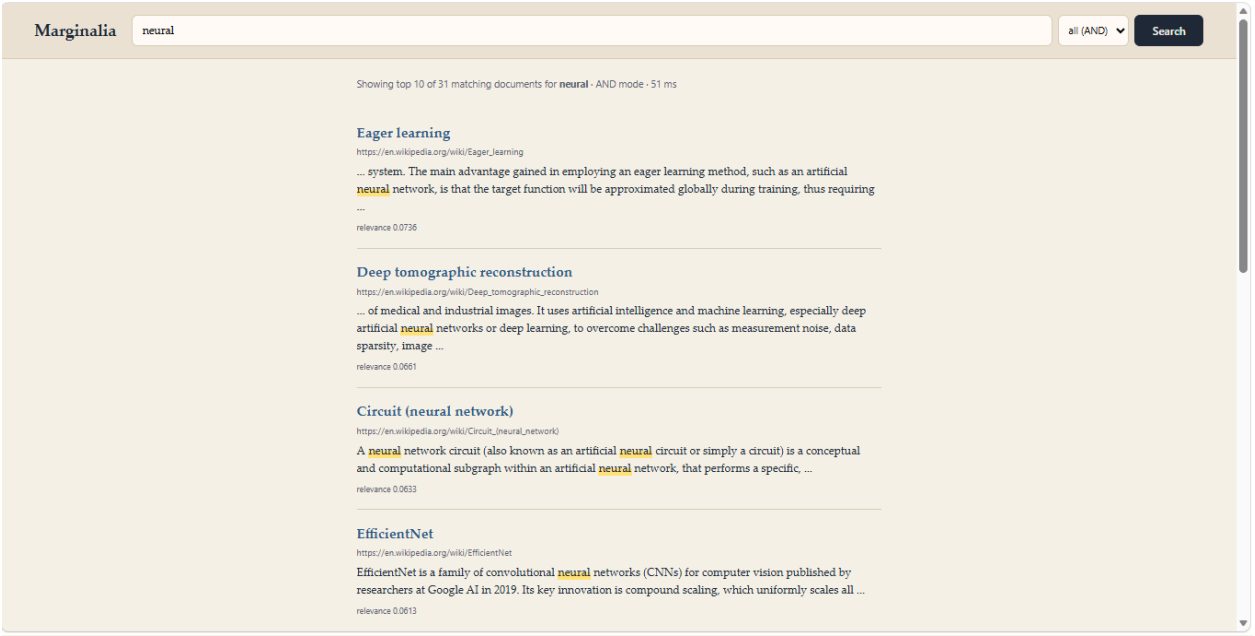

Figure 5: Single-word query "neural" in AND mode — also 31 documents, identical to OR as expected.

5.4 Conclusion

This project successfully implemented a specialized educational search engine from scratch, including a hand-built inverted index, Boolean (AND/OR) query processing, TF-IDF ranking with document-length normalisation, and a clean web interface with snippet generation and keyword highlighting, without using any pre-built search library. Testing against a 156-document corpus confirmed that the system returns relevant, sensibly ranked results in around 50 milliseconds, and that the Boolean modes behave

correctly. The main lessons were the importance of cleaning collected data, the value of normalising both queries and documents identically, and the practical trade-offs between retrieval accuracy and efficiency. The limitations identified above, particularly corpus cleaning and the bag-of-words assumption, point to clear and achievable directions for future improvement.